Python Functions

This tutorial will go over the fundamentals of Python functions, including what they are, their syntax, their primary parts, return keywords, and significant types. We'll also look at some examples of Python function definitions.

What are Python Functions?

A collection of related assertions that carry out a mathematical, analytical, or evaluative operation is known as a function. An assortment of proclamations called Python Capabilities returns the specific errand. Python functions are necessary for intermediate-level programming and are easy to define. Function names meet the same standards as variable names do. The objective is to define a function and group-specific frequently performed actions. Instead of repeatedly creating the same code block for various input variables, we can call the function and reuse the code it contains with different variables.

Client-characterized and worked-in capabilities are the two primary classes of capabilities in Python. It aids in maintaining the program's uniqueness, conciseness, and structure.

Advantages of Python Functions

Pause We can stop a program from repeatedly using the same code block by including functions.

- Once defined, Python functions can be called multiple times and from any location in a program.
- Our Python program can be broken up into numerous, easy-to-follow functions if it is significant.
- The ability to return as many outputs as we want using a variety of arguments is one of Python's most significant achievements.
- However, Python programs have always incurred overhead when calling functions.

However, calling functions has always been overhead in a Python program.

Syntax

```
#  An example Python Function
def function_name( parameters ):
    # code block
```

The accompanying components make up to characterize a capability, as seen previously.

- The start of a capability header is shown by a catchphrase called def.

- function_name is the function's name, which we can use to distinguish it from other functions. We will utilize this name to call the capability later in the program. Name functions in Python must adhere to the same guidelines as naming variables.
- Using parameters, we provide the defined function with arguments. Notwithstanding, they are discretionary.
- A colon (:) marks the function header's end.
- We can utilize a documentation string called docstring in the short structure to make sense of the reason for the capability.
- Several valid Python statements make up the function's body. The entire code block's indentation depth-typically four spaces-must be the same.
- A return expression can get a value from a defined function.

Illustration of a User-Defined Function

We will define a function that returns the argument number's square when called.

```
# Example Python Code for User-Defined function
def square( num ):
    """
    This function computes the square of the number.
    """
    return num**2
object_ = square(6)
print( "The square of the given number is: ", object_ )
```

Output:

```
The square of the given number is:  36
```

Calling a Function

Calling a Function To define a function, use the def keyword to give it a name, specify the arguments it must receive, and organize the code block.

When the fundamental framework for a function is finished, we can call it from anywhere in the program. An illustration of how to use the a_function function can be found below.

```
# Example Python Code for calling a function
# Defining a function
def a_function( string ):
```

```
    "This prints the value of length of string"
    return len(string)

# Calling the function we defined
print( "Length of the string Functions is: ", a_function( "Functions" ) )
print( "Length of the string Python is: ", a_function( "Python" ) )
```

Output:

```
Length of the string Functions is:  9
Length of the string Python is:  6
```

Pass by Reference vs. Pass by Value

In the Python programming language, all parameters are passed by reference. It shows that if we modify the worth of contention within a capability, the calling capability will similarly mirror the change. For instance,

Code

```
# Example Python Code for Pass by Reference vs. Value
# defining the function
def square( item_list ):
    """This function will find the square of items in the list"'
    squares = [ ]
    for l in item_list:
        squares.append( l**2 )
    return squares

# calling the defined function
my_list = [17, 52, 8];
my_result = square( my_list )
print( "Squares of the list are: ", my_result )
```

Output:

```
Squares of the list are:  [289, 2704, 64]
```

Function Arguments

The following are the types of arguments that we can use to call a function:

1. Default arguments
2. Keyword arguments
3. Required arguments
4. Variable-length arguments

1) Default Arguments

A default contention is a boundary that takes as information a default esteem, assuming that no worth is provided for the contention when the capability is called. The following example demonstrates default arguments.

Code

```
# Python code to demonstrate the use of default arguments
# defining a function
def function( n1, n2 = 20 ):
    print("number 1 is: ", n1)
    print("number 2 is: ", n2)


# Calling the function and passing only one argument
print( "Passing only one argument" )
function(30)

# Now giving two arguments to the function
print( "Passing two arguments" )
function(50,30)
```

Output:

```
Passing only one argument
number 1 is:  30
number 2 is:  20
Passing two arguments
number 1 is:  50
number 2 is:  30
```

2) Keyword Arguments

Keyword arguments are linked to the arguments of a called function. While summoning a capability with watchword contentions, the client might tell whose boundary esteem it is by looking at the boundary name.

We can eliminate or orchestrate specific contentions in an alternate request since the Python translator will interface the furnished watchwords to connect the qualities with its boundaries. One more method for utilizing watchwords to summon the capability() strategy is as per the following:

Code

```
# Python code to demonstrate the use of keyword arguments
  # Defining a function
def function( n1, n2 ):
    print("number 1 is: ", n1)
    print("number 2 is: ", n2)

# Calling function and passing arguments without using keyword
print( "Without using keyword" )
function( 50, 30)

# Calling function and passing arguments using keyword
print( "With using keyword" )
function( n2 = 50, n1 = 30)
```

Output:

```
Without using keyword
number 1 is:  50
number 2 is:  30
With using keyword
number 1 is:  30
number 2 is:  50
```

3) Required Arguments

Required arguments are those supplied to a function during its call in a predetermined positional sequence. The number of arguments required in the method call must be the same as those provided in the function's definition.

We should send two contentions to the capability() all put together; it will return a language structure blunder, as seen beneath.

Code

```
# Python code to demonstrate the use of default arguments
# Defining a function
def function( n1, n2 ):
    print("number 1 is: ", n1)
    print("number 2 is: ", n2)

# Calling function and passing two arguments out of order, we need num1 to be 20 an
d num2 to be 30
print( "Passing out of order arguments" )
function( 30, 20 )

# Calling function and passing only one argument
print( "Passing only one argument" )
try:
    function( 30 )
except:
    print( "Function needs two positional arguments" )
```

Output:

```
Passing out of order arguments
number 1 is:  30
number 2 is:  20
Passing only one argument
Function needs two positional arguments
```

4) Variable-Length Arguments

We can involve unique characters in Python capabilities to pass many contentions. However, we need a capability. This can be accomplished with one of two types of characters:

"args" and "kwargs" refer to arguments not based on keywords.

To help you understand arguments of variable length, here's an example.

Code

```
# Python code to demonstrate the use of variable-length arguments
# Defining a function
def function( *args_list ):
```

```
    ans = []
    for l in args_list:
        ans.append( l.upper() )
    return ans
# Passing args arguments
object = function('Python', 'Functions', 'tutorial')
print( object )

# defining a function
def function( **kargs_list ):
    ans = []
    for key, value in kargs_list.items():
        ans.append([key, value])
    return ans
# Paasing kwargs arguments
object = function(First = "Python", Second = "Functions", Third = "Tutorial")
print(object)
```

Output:

```
['PYTHON', 'FUNCTIONS', 'TUTORIAL']
[['First', 'Python'], ['Second', 'Functions'], ['Third', 'Tutorial']]
```

return Statement

When a defined function is called, a return statement is written to exit the function and return the calculated value.

Syntax:

```
return < expression to be returned as output >
```

The return statement can be an argument, a statement, or a value, and it is provided as output when a particular job or function is finished. A declared function will return an empty string if no return statement is written.

A return statement in Python functions is depicted in the following example.

Code

```
# Python code to demonstrate the use of return statements
# Defining a function with return statement
```

```
def square( num ):
    return num**2

# Calling function and passing arguments.
print( "With return statement" )
print( square( 52 ) )

# Defining a function without return statement
def square( num ):
    num**2

# Calling function and passing arguments.
print( "Without return statement" )
print( square( 52 ) )
```

Output:

```
With return statement
2704
Without return statement
None
```

The Anonymous Functions

Since we do not use the def keyword to declare these kinds of Python functions, they are unknown. The lambda keyword can define anonymous, short, single-output functions.

Arguments can be accepted in any number by lambda expressions; However, the function only produces a single value from them. They cannot contain multiple instructions or expressions. Since lambda needs articulation, a mysterious capability can't be straightforwardly called to print.

Lambda functions can only refer to variables in their argument list and the global domain name because they contain their distinct local domain.

In contrast to inline expressions in C and C++, which pass function stack allocations at execution for efficiency reasons, lambda expressions appear to be one-line representations of functions.

Syntax

Lambda functions have exactly one line in their syntax:

```
lambda [argument1 [,argument2... .argumentn]] : expression
```

Below is an illustration of how to use the lambda function:

Code

```
# Python code to demonstrate ananymous functions
# Defining a function
lambda_ = lambda argument1, argument2: argument1 + argument2;

# Calling the function and passing values
print( "Value of the function is : ", lambda_( 20, 30 ) )
print( "Value of the function is : ", lambda_( 40, 50 ) )
```

Output:

```
Value of the function is :  50
Value of the function is :  90
```

Scope and Lifetime of Variables

A variable's scope refers to the program's domain wherever it is declared. A capability's contentions and factors are not external to the characterized capability. They only have a local domain as a result.

The length of time a variable remains in RAM is its lifespan. The lifespan of a function is the same as the lifespan of its internal variables. When we exit the function, they are taken away from us. As a result, the value of a variable in a function does not persist from previous executions.

An easy illustration of a function's scope for a variable can be found here.

Code

```
# Python code to demonstrate scope and lifetime of variables
#defining a function to print a number.
def number( ):
    num = 50
    print( "Value of num inside the function: ", num)
num = 10
number()
print( "Value of num outside the function:", num)
```

Output:

```
Value of num inside the function:  50
```

Value of num outside the function: 10

Here, we can see that the initial value of num is 10. Even though the function number() changed the value of num to 50, the value of num outside of the function remained unchanged.

This is because the capability's interior variable num is not quite the same as the outer variable (nearby to the capability). Despite having a similar variable name, they are separate factors with discrete extensions.

Factors past the capability are available inside the capability. The impact of these variables is global. We can retrieve their values within the function, but we cannot alter or change them. The value of a variable can be changed outside of the function if it is declared global with the keyword global.

Python Capability inside Another Capability

Capabilities are viewed as top-of-the-line objects in Python. First-class objects are treated the same everywhere they are used in a programming language. They can be stored in built-in data structures, used as arguments, and in conditional expressions. If a programming language treats functions like first-class objects, it is considered to implement first-class functions. Python lends its support to the concept of First-Class functions.

A function defined within another is called an "inner" or "nested" function. The parameters of the outer scope are accessible to inner functions. Internal capabilities are developed to cover them from the progressions outside the capability. Numerous designers see this interaction as an embodiment.

Code

```
# Python code to show how to access variables of a nested functions
# defining a nested function
def word():
    string = 'Python functions tutorial'
    x = 5
    def number():
        print( string )
        print( x )

    number()
word()
```

Output:

Python functions tutorial
5

Python Modules

In this tutorial, we will explain how to construct and import custom Python modules. Additionally, we may import or integrate Python's built-in modules via various methods.

What is Modular Programming?

Modular programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub-tasks. We call these subtasks modules. Therefore, we can build a bigger program by assembling different modules that act like building blocks.

Modularizing our code in a big application has a lot of benefits.

Simplification: A module often concentrates on one comparatively small area of the overall problem instead of the full task. We will have a more manageable design problem to think about if we are only concentrating on one module. Program development is now simpler and much less vulnerable to mistakes.

Flexibility: Modules are frequently used to establish conceptual separations between various problem areas. It is less likely that changes to one module would influence other portions of the program if modules are constructed in a fashion that reduces interconnectedness. (We might even be capable of editing a module despite being familiar with the program beyond it.) It increases the likelihood that a group of numerous developers will be able to collaborate on a big project.

Reusability: Functions created in a particular module may be readily accessed by different sections of the assignment (through a suitably established api). As a result, duplicate code is no longer necessary.

Scope: Modules often declare a distinct namespace to prevent identifier clashes in various parts of a program.

In Python, modularization of the code is encouraged through the use of functions, modules, and packages.

What are Modules in Python?

A document with definitions of functions and various statements written in Python is called a Python module.

In Python, we can define a module in one of 3 ways:

- Python itself allows for the creation of modules.
- Similar to the re (regular expression) module, a module can be primarily written in C programming language and then dynamically inserted at run-time.
- A built-in module, such as the itertools module, is inherently included in the interpreter.

A module is a file containing Python code, definitions of functions, statements, or classes. An example_module.py file is a module we will create and whose name is example_module.

We employ modules to divide complicated programs into smaller, more understandable pieces. Modules also allow for the reuse of code.

Rather than duplicating their definitions into several applications, we may define our most frequently used functions in a separate module and then import the complete module.

Let's construct a module. Save the file as example_module.py after entering the following.

Example:

```
# Here, we are creating a simple Python program to show how to create a module.
# defining a function in the module to reuse it
def square( number ):
   # here, the above function will square the number passed as the input
    result = number ** 2
    return result     # here, we are returning the result of the function
```

Here, a module called example_module contains the definition of the function square(). The function returns the square of a given number.

How to Import Modules in Python?

In Python, we may import functions from one module into our program, or as we say into, another module.

For this, we make use of the import Python keyword. In the Python window, we add the next to import keyword, the name of the module we need to import. We will import the module we defined earlier example_module.

Syntax:

```
import example_module
```

The functions that we defined in the example_module are not immediately imported into the present program. Only the name of the module, i.e., example_ module, is imported here.

We may use the dot operator to use the functions using the module name. For instance:

Example:

```
# here, we are calling the module square method and passing the value 4
result = example_module.square(  4  )
print("By using the module square of number is: ", result )
```

Output:

```
By using the module square of number is: 16
```

There are several standard modules for Python. The complete list of Python standard modules is available. The list can be seen using the help command.

Similar to how we imported our module, a user-defined module, we can use an import statement to import other standard modules.

Importing a module can be done in a variety of ways. Below is a list of them.

Python import Statement

Using the import Python keyword and the dot operator, we may import a standard module and can access the defined functions within it. Here's an illustration.

Code

```
# Here, we are creating a simple Python program to show how to import a standard m
odule
# Here, we are import the math module which is a standard module
import math
print( "The value of euler's number is", math.e )
# here, we are printing the euler's number from the math module
```

Output:

```
The value of euler's number is 2.718281828459045
```

Importing and also Renaming

While importing a module, we can change its name too. Here is an example to show.

Code

```
# Here, we are creating a simple Python program to show how to import a module and
 rename it
# Here, we are import the math module and give a different name to it
import math as mt     # here, we are importing the math module as mt
print( "The value of euler's number is", mt.e )
# here, we are printing the euler's number from the math module
```

Output:

```
The value of euler's number is 2.718281828459045
```

The math module is now named mt in this program. In some situations, it might help us type faster in case of modules having long names.

Please take note that now the scope of our program does not include the term math. Thus, mt.pi is the proper implementation of the module, whereas math.pi is invalid.

Python from...import Statement

We can import specific names from a module without importing the module as a whole. Here is an example.

Code

```
# Here, we are creating a simple Python program to show how to import specific
# objects from a module
# Here, we are import euler's number from the math module using the from keyword

from math import e
# here, the e value represents the euler's number
print( "The value of euler's number is", e )
```

Output:

```
The value of euler's number is 2.718281828459045
```

Only the e constant from the math module was imported in this case.

We avoid using the dot (.) operator in these scenarios. As follows, we may import many attributes at the same time:

Code

```
# Here, we are creating a simple Python program to show how to import multiple
# objects from a module
from math import e, tau
print( "The value of tau constant is: ", tau )
print( "The value of the euler's number is: ", e )
```

Output:

```
The value of tau constant is:  6.283185307179586
The value of the euler's number is:  2.718281828459045
```

Import all Names - From import * Statement

To import all the objects from a module within the present namespace, use the * symbol and the from and import keyword.

Syntax:

1. **from** name_of_module **import** *

There are benefits and drawbacks to using the symbol *. It is not advised to use * unless we are certain of our particular requirements from the module; otherwise, do so.

Here is an example of the same.

Code

```
# Here, we are importing the complete math module using *
from math import *
# Here, we are accessing functions of math module without using the dot operator
print( "Calculating square root: ", sqrt(25) )
# here, we are getting the sqrt method and finding the square root of 25
print( "Calculating tangent of an angle: ", tan(pi/6) )
# here pi is also imported from the math module
```

Output:

```
Calculating square root:  5.0
Calculating tangent of an angle:  0.5773502691896257
```

Locating Path of Modules

The interpreter searches numerous places when importing a module in the Python program. Several directories are searched if the built-in module is not present. The list of directories can be accessed using sys.path. The Python interpreter looks for the module in the way described below:

The module is initially looked for in the current working directory. Python then explores every directory in the shell parameter PYTHONPATH if the module cannot be located in the current directory. A list of folders makes up the environment variable known as PYTHONPATH. Python examines the installation-dependent set of folders set up when Python is downloaded if that also fails.

Here is an example to print the path.

Code

```
# Here, we are importing the sys module
import sys
# Here, we are printing the path using sys.path
print("Path of the sys module in the system is:", sys.path)
```

Output:

```
Path of the sys module in the system is:
['/home/pyodide', '/home/pyodide/lib/Python310.zip', '/lib/Python3.10', '/lib/Python3.10/lib-dynload', '', '/lib/Python3.10/site-packages']
```

The dir() Built-in Function

We may use the dir() method to identify names declared within a module.

For instance, we have the following names in the standard module str. To print the names, we will use the dir() method in the following way:

Code

```
# Here, we are creating a simple Python program to print the directory of a module
print( "List of functions:\n ", dir( str ), end=", " )
```

Output:

```
List of functions:
 ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

Namespaces and Scoping

Objects are represented by names or identifiers called variables. A namespace is a dictionary containing the names of variables (keys) and the objects that go with them (values).

Both local and global namespace variables can be accessed by a Python statement. When two variables with the same name are local and global, the local variable takes the role of the global variable. There is a separate local namespace for every function. The scoping rule for class methods is the same as for regular functions. Python determines if parameters are local or

global based on reasonable predictions. Any variable that is allocated a value in a method is regarded as being local.

Therefore, we must use the global statement before we may provide a value to a global variable inside of a function. Python is informed that Var_Name is a global variable by the line global Var_Name. Python stops looking for the variable inside the local namespace.

We declare the variable Number, for instance, within the global namespace. Since we provide a Number a value inside the function, Python considers a Number to be a local variable. UnboundLocalError will be the outcome if we try to access the value of the local variable without or before declaring it global.

Code

```
1. Number = 204
2. def AddNumber():  # here, we are defining a function with the name Add Number
3.     # Here, we are accessing the global namespace
4.     global Number
5.     Number = Number + 200
6. print("The number is:", Number)
7. # here, we are printing the number after performing the addition
8. AddNumber()    # here, we are calling the function
9. print("The number is:", Number)
```

Output:

```
The number is: 204
The number is: 404
```

Difference between Modules and Packages in Python

It is often that many coders and amateur programmers may confuse between a module and a package. The problem generally arises when it becomes hard to identify when and where a module or a package should be implemented.

In the following tutorial, we will discuss a clear set of **differences in modules and packages in the Python programming language** that will make it easy for the programmers to work more professionally while dealing with both modules and packages.

Understanding the Modules in Python

A **module** is a pythonic statement containing different functions. Modules act as a pre-defined library in the script, which is accessible to both the programmers as well as the user.

The python modules also store pre-defined functions from the library when the code is being executed.

Let us consider an example demonstrating the use of a module in Python

Example:

```
# importing the library and module
import math
from math import pow
# using the pow() function
pow(3, 5)
# printing pow()
print(pow)
```

Output:

```
<built-in function pow>
```

Explanation:

In the above snippet of code, we have imported the required module and used the **pow()** function to calculate the powers of the given number as arguments. We have then printed the value of the **pow** for the user.

Understanding the Packages in Python

A **package** is considered a collection of tools that allows the programmers to initiate the code. A Python package acts as a user-variable interface for any source code. This feature allows a Python package to work at a defined time for any functional script in the runtime.

Let us consider the following example demonstrating the package in Python.

Example:

```
# importing the package
import math
# printing a statement
print("We have imported the math package")
```

Output:

```
We have imported the math package
```

Explanation:

In the above snippet of code, we have imported the **math** package that consists of various modules and functions for the programmers and printed a statement for the users.

Understanding the differences between Python Modules and Packages

1. A Package consists of the **__init__.py** file for each user-oriented script. However, the same does not apply to the modules in runtime for any script specified to the users.
2. A module is a file that contains a Python script in runtime for the code specified to the users. A package also modifies the user interpreted code in such a manner that it gets easily operated in the runtime.

A python "module" contains a unit namespace, with the variables that are extracted locally along with some parsed functions like:

a. Constants and Variables
b. Any old or new value
c. Class definitions of properties
d. A module generally corresponds to a single file
e. A debugging tool in the user interface library.

There are some usually utilized tools that allow the programmers to build a new platform with the help of the modules for better code executions. This installs and distributes packages throughout the library in runtime as well.

It becomes easier for us to employ the user-specific tools with the help of a well-structured and standard layout for the package.